Katalon Studio Static Code Analysis Setup Guide

This document provides the complete setup for running static code analysis in **Katalon Studio v10** using **Java 17**, **Gradle 8.10**, and **CodeNarc v3.4.0**.

---

1. Prerequisites

Before beginning the setup, ensure your system is configured with the following:

- **Java Version:** OpenJDK 17, must be installed.
- **Katalon Version:** This guide is optimized for **Katalon Studio v10**.
- **Project Structure:** Ensure you have a project folder (e.g., `staticCodeAnalysis`) with a `Keywords` folder containing your `.groovy` files.

---

2. Configuration Files

Create the following two files in the root of your `staticCodeAnalysis` folder.

A. build.gradle

This file manages the dependencies and the scan task.

```
plugins {
    id 'groovy'
    id 'codenarc'
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.codehaus.groovy:groovy-all:3.0.17'
}

codenarc {
    toolVersion = '3.4.0'
    configFile = file("${project.rootDir}/codenarc.ruleset")
```

```
    reportFormat = 'html'
}

tasks.withType(CodeNarc) {
    // Specifically targets Katalon Keyword folders
    source = fileTree(dir: 'Keywords', include: '**/*.groovy')
    ignoreFailures = true
}
```

B. codenarc.ruleset

This file defines which coding rules to enforce. This version is verified to avoid common "No such rule" errors.

```
ruleset {
   description 'Stable Enhanced Ruleset for Katalon'
   // --- 1. Standard Rule Packages (Guaranteed stable) ---
   // These load all rules within these categories that exist in your version
   ruleset('rulesets/basic.xml')
   ruleset('rulesets/braces.xml')
   ruleset('rulesets/concurrency.xml')
   ruleset('rulesets/convention.xml')
   ruleset('rulesets/design.xml')
   ruleset('rulesets/dry.xml')
   ruleset('rulesets/exceptions.xml')
   ruleset('rulesets/formatting.xml')
   ruleset('rulesets/generic.xml')
   ruleset('rulesets/groovyism.xml')
   ruleset('rulesets/imports.xml')
   ruleset('rulesets/logging.xml')
   ruleset('rulesets/naming.xml')
   ruleset('rulesets/security.xml')
   ruleset('rulesets/serialization.xml')
   ruleset('rulesets/size.xml')
   ruleset('rulesets/unnecessary.xml')
   ruleset('rulesets/unused.xml')
   // --- 2. Custom Rule Configurations ---

   LineLength {
      length = 150  // Supported in all modern versions
   }
   ClassName {
      priority = 1  // Supported in all modern versions
```

```
  }
  // We removed the Regex parameter to ensure the build passes.
  // This will now use the default CodeNarc behavior for empty catches.
  EmptyCatchBlock {
     enabled = true
  }
}
```

3. Setup and Execution Commands

Run these commands in your terminal from inside the project folder.

Step 1: Set the Java Environment Ensure your terminal session is pointed to Java 17.

```
export JAVA_HOME=$(/usr/libexec/java_home -v 17)
```

Run the below commands in your Katalon project directory:

Step 2: Initialize the Gradle Wrapper Generate the infrastructure for Gradle 8.10.

```
gradle wrapper --gradle-version 8.10
```

Step 3: Run the Analysis Execute the scan to generate your report inside your katalon project:

```
./gradlew clean codenarcMain
```

4. Viewing the Results

- **Report Location:** `build/reports/codenarc/main.html`.
- **Summary Data:** The report provides a count of total files, files with violations, and a breakdown of priorities (1, 2, and 3).
- **How to Open:** Use the command `open build/reports/codenarc/main.html` to view the results in your browser.

Example Report:

File /Users/bhavyansh-katalon/Katalon%20Studio/staticCodeAnalysis/build/reports/codenarc/main.html

CodeNarc Report

Report title:	
Date:	12-Feb-2026, 2:40:00 pm
Generated with:	CodeNarc v3.4.0

CODENARC
Less Bugs Better Code

Summary

Total Files	Files with Violations	Priority 1	Priority 2	Priority 3
1	1	-	7	25

Violations

Sort by Rule Name | Sort by Rule Priority | Sort by Rule (w/Most Violations) | Sort by File (w/Most Violations)

File	Rule Name	Priority	Line #	Source Line / Message
MySampleKeyword.groovy	CompileStatic	2	6	[SRC] class MySampleKeyword { [MSG] *Class should be marked with one of @GrailsCompileStatic, @CompileStatic or @CompileDynamic*
MySampleKeyword.groovy	Println	2	12	[SRC] println("Checking: " + text) [MSG] *println should be replaced with something more robust*
MySampleKeyword.groovy	EmptyCatchBlock	2	13	[SRC] } catch (Exception e) { [MSG] *The catch block is empty*
MySampleKeyword.groovy	CatchException	2	13	[SRC] } catch (Exception e) { [MSG] *The type Exception should not be caught*